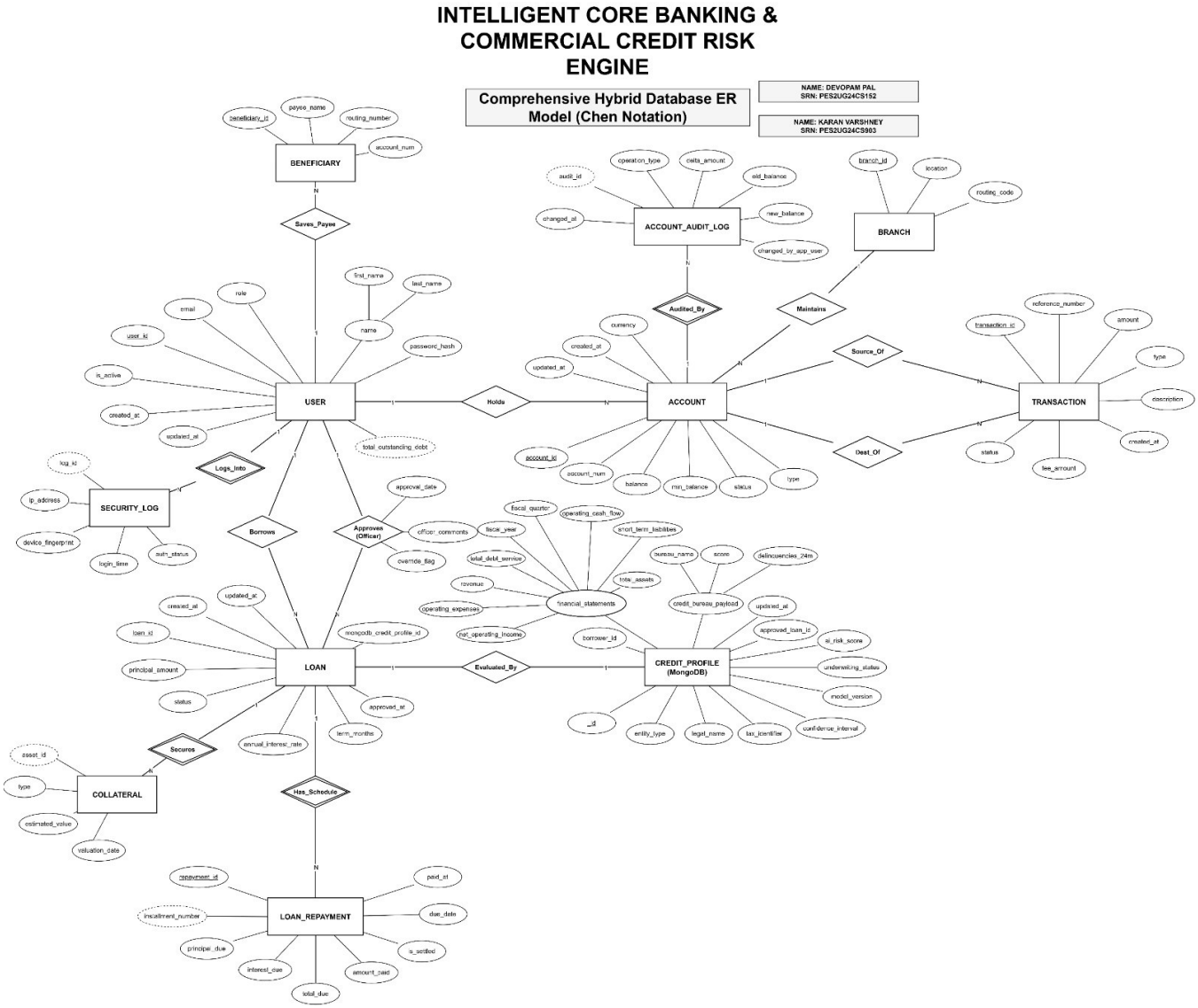
INTELLIGENT CORE BANKING & COMMERCIAL CREDIT RISK ENGINE

Relational Schema, Mapping & MySQL DDL



Based on the submitted Chen-notation ER diagram

1. How to Read the Relational Schema

The relational schema is the table form of the ER diagram. Each entity normally becomes a table, each key attribute becomes a primary key, and relationships are represented using foreign keys.

Main rule: In a 1:N relationship, the primary key of the 1-side is placed as a foreign key in the N-side.

Important conversion rules:

ER concept	Relational conversion
Strong entity	Entity becomes a table; key attribute becomes PRIMARY KEY.
Composite attribute	Store the component attributes. Example: name becomes first_name and last_name.
Derived attribute	Usually do not store it; calculate it when required.
1:N relationship	Put the primary key of the 1-side as a foreign key in the N-side.
Same entity in two roles	Use two foreign keys with different names, e.g. source_account_id and destination_account_id.
Relationship attributes	For a simple 1:N relationship, relationship attributes can be stored on the N-side.

2. Complete Relational Schema

The schema below shows each relation with its primary key, foreign keys and remaining attributes. This format is easier to read and directly matches the ER-to-relational conversion.

A. Core Users, Accounts and Access

Relation	Primary Key	Foreign Key(s)	Other Attributes
USER	user_id	-	email, first_name, last_name, password_hash, role, is_active, created_at, updated_at
BRANCH	branch_id	-	location, routing_code
ACCOUNT	account_id	user_id -> USER; branch_id -> BRANCH	account_num, balance, min_balance, status, type, currency, created_at, updated_at
BENEFICIARY	beneficiary_id	user_id -> USER	payee_name, routing_number, account_num
SECURITY_LOG	log_id	user_id -> USER	ip_address, device_fingerprint, login_time, auth_status

B. Transactions, Loans and Audit

Relation	Primary Key	Foreign Key(s)	Other Attributes
TRANSACTION	transaction_id	source_account_id -> ACCOUNT; destination_account_id -> ACCOUNT	reference_number, amount, type, description, status, fee_amount, created_at
ACCOUNT_AUDIT_LOG	audit_id	account_id -> ACCOUNT	operation_type, delta_amount, old_balance, new_balance, changed_by_app_user, changed_at
LOAN	loan_id	borrower_id -> USER; officer_id -> USER	mongodb_credit_profile_id, principal_amount, annual_interest_rate, term_months, status, approval_date, officer_comments, override_flag, approved_at, created_at, updated_at
COLLATERAL	asset_id	loan_id -> LOAN	type, estimated_value, valuation_date

LOAN_REPAYMENT	repayment_id	loan_id -> LOAN	installment_number, principal_due, interest_due, total_due, amount_paid, due_date, paid_at, is_settled

Compact relational notation

```
USER(user_id PK, email, first_name, last_name, password_hash, role, is_active, created_at, updated_at)
BRANCH(branch_id PK, location, routing_code)
ACCOUNT(account_id PK, user_id FK, branch_id FK, account_num, balance, min_balance, status, type, currency, created_at, updated_at)
BENEFICIARY(beneficiary_id PK, user_id FK, payee_name, routing_number, account_num)
SECURITY_LOG(log_id PK, user_id FK, ip_address, device_fingerprint, login_time, auth_status)
TRANSACTION(transaction_id PK, source_account_id FK, destination_account_id FK, reference_number, amount, type, description, status, fee_amount, created_at)
ACCOUNT_AUDIT_LOG(audit_id PK, account_id FK, operation_type, delta_amount, old_balance, new_balance, changed_by_app_user, changed_at)
LOAN(loan_id PK, borrower_id FK, officer_id FK, mongodb_credit_profile_id, principal_amount, annual_interest_rate, term_months, status, approval_date, officer_comments, override_flag, approved_at, created_at, updated_at)
COLLATERAL(asset_id PK, loan_id FK, type, estimated_value, valuation_date)
LOAN_REPAYMENT(repayment_id PK, loan_id FK, installment_number, principal_due, interest_due, total_due, amount_paid, due_date, paid_at, is_settled)
```

PK = Primary Key | FK = Foreign Key. The arrows in the table show which parent relation each foreign key references.

3. Table-by-Table Schema Description

USER

Stores the people who use the banking system. A user can be a customer, loan officer, or risk administrator. The ER diagram shows name as a composite attribute, so it is split into first_name and last_name. The derived total_outstanding_debt is not stored because it can be calculated from the user's loans.

Primary key: user_id.

BRANCH

Stores bank branch details. A branch may maintain many accounts.

Primary key: branch_id.

ACCOUNT

Stores every bank account. The user_id foreign key tells us who owns the account, and branch_id tells us which branch maintains it. Because USER-Holds-ACCOUNT and BRANCH-Maintains-ACCOUNT are 1:N relationships, both foreign keys are placed in ACCOUNT.

Primary key: account_id. Foreign keys: user_id, branch_id.

BENEFICIARY

Stores saved payees of a user. One user can save many beneficiaries, so user_id is stored in BENEFICIARY.

Primary key: beneficiary_id. Foreign key: user_id.

SECURITY_LOG

Stores login activity such as IP address, device fingerprint, login time, and authentication status. A user may generate many login records, therefore user_id is a foreign key here.

Primary key: log_id. Foreign key: user_id.

TRANSACTION

Stores deposits, withdrawals, transfers, repayments, and other banking transactions. The same ACCOUNT entity participates twice: once as source and once as destination. Therefore the table uses two foreign keys: source_account_id and destination_account_id.

Primary key: transaction_id. Foreign keys: source_account_id, destination_account_id.

ACCOUNT_AUDIT_LOG

Stores a history of account balance changes. One account can have many audit records. The old balance, new balance and delta amount make it possible to trace every change.

Primary key: audit_id. Foreign key: account_id.

LOAN

Stores every loan application or approved loan. borrower_id identifies the customer who borrowed the money, while officer_id identifies the user acting as the loan officer. The approval relationship attributes approval_date, officer_comments and override_flag are stored in the LOAN table because one officer can approve many loans.

Primary key: loan_id. Foreign keys: borrower_id, officer_id.

COLLATERAL

Stores assets pledged against a loan, such as property, land or a vehicle. One loan can have multiple collateral assets, so loan_id is a foreign key in COLLATERAL.

Primary key: asset_id. Foreign key: loan_id.

LOAN_REPAYMENT

Stores the repayment schedule of each loan. One loan can have many installments. The combination of loan_id and installment_number should be unique so the same installment number cannot occur twice for one loan.

Primary key: repayment_id. Foreign key: loan_id.

4. Relationship-to-Foreign-Key Mapping

Relationship	Cardinality	Relational implementation
USER - Holds - ACCOUNT	1:N	user_id placed in ACCOUNT
USER - Saves_Payee - BENEFICIARY	1:N	user_id placed in BENEFICIARY
USER - Logs_Into - SECURITY_LOG	1:N	user_id placed in SECURITY_LOG
USER - Borrows - LOAN	1:N	borrower_id placed in LOAN
USER - Approves - LOAN	1:N	officer_id placed in LOAN
BRANCH - Maintains - ACCOUNT	1:N	branch_id placed in ACCOUNT
ACCOUNT - Source_Of - TRANSACTION	1:N	source_account_id placed in TRANSACTION
ACCOUNT - Dest_Of - TRANSACTION	1:N	destination_account_id placed in TRANSACTION
ACCOUNT - Audited_By - ACCOUNT_AUDIT_LOG	1:N	account_id placed in ACCOUNT_AUDIT_LOG
LOAN - Secures - COLLATERAL	1:N	loan_id placed in COLLATERAL
LOAN - Has_Schedule - LOAN_REPAYMENT	1:N	loan_id placed in LOAN_REPAYMENT

5. MySQL DDL Commands

The following DDL uses simple MySQL 8.x syntax. The table name bank_transaction is used instead of transaction to avoid confusion with the SQL TRANSACTION keyword.

```
CREATE DATABASE intelligent_banking;
USE intelligent_banking;

CREATE TABLE users (
    user_id INT AUTO_INCREMENT,
    email VARCHAR(100) NOT NULL UNIQUE,
    first_name VARCHAR(50) NOT NULL,
    last_name VARCHAR(50) NOT NULL,
    password_hash VARCHAR(255) NOT NULL,
    role VARCHAR(30) NOT NULL,
    is_active BOOLEAN DEFAULT TRUE,
    created_at DATETIME DEFAULT CURRENT_TIMESTAMP,
    updated_at DATETIME DEFAULT CURRENT_TIMESTAMP
        ON UPDATE CURRENT_TIMESTAMP,
    PRIMARY KEY (user_id)
);

CREATE TABLE branch (
    branch_id INT AUTO_INCREMENT,
    location VARCHAR(100) NOT NULL,
    routing_code VARCHAR(30) NOT NULL UNIQUE,
    PRIMARY KEY (branch_id)
);

CREATE TABLE account (
    account_id INT AUTO_INCREMENT,
    user_id INT NOT NULL,
    branch_id INT NOT NULL,
    account_num VARCHAR(30) NOT NULL UNIQUE,
    balance DECIMAL(15,2) DEFAULT 0.00,
    min_balance DECIMAL(15,2) DEFAULT 0.00,
    status VARCHAR(20) NOT NULL,
    type VARCHAR(30) NOT NULL,
    currency VARCHAR(10) DEFAULT 'INR',
    created_at DATETIME DEFAULT CURRENT_TIMESTAMP,
    updated_at DATETIME DEFAULT CURRENT_TIMESTAMP
        ON UPDATE CURRENT_TIMESTAMP,
    PRIMARY KEY (account_id),
    FOREIGN KEY (user_id) REFERENCES users(user_id),
    FOREIGN KEY (branch_id) REFERENCES branch(branch_id)
);

CREATE TABLE beneficiary (
    beneficiary_id INT AUTO_INCREMENT,
    user_id INT NOT NULL,
    payee_name VARCHAR(100) NOT NULL,
    routing_number VARCHAR(30),
    account_num VARCHAR(30) NOT NULL,
    PRIMARY KEY (beneficiary_id),
    FOREIGN KEY (user_id) REFERENCES users(user_id)
);

CREATE TABLE security_log (
    log_id INT AUTO_INCREMENT,
    user_id INT NOT NULL,
    ip_address VARCHAR(45),
    device_fingerprint VARCHAR(255),
    login_time DATETIME DEFAULT CURRENT_TIMESTAMP,
    auth_status VARCHAR(30),
    PRIMARY KEY (log_id),
    FOREIGN KEY (user_id) REFERENCES users(user_id)
);
```

```
CREATE TABLE bank_transaction (
    transaction_id INT AUTO_INCREMENT,
    source_account_id INT,
    destination_account_id INT,
    reference_number VARCHAR(50) NOT NULL UNIQUE,
    amount DECIMAL(15,2) NOT NULL,
    type VARCHAR(30) NOT NULL,
    description VARCHAR(255),
    status VARCHAR(30),
    fee_amount DECIMAL(15,2) DEFAULT 0.00,
    created_at DATETIME DEFAULT CURRENT_TIMESTAMP,
    PRIMARY KEY (transaction_id),
    FOREIGN KEY (source_account_id) REFERENCES account(account_id),
    FOREIGN KEY (destination_account_id) REFERENCES account(account_id),
    CHECK (amount > 0)
);

CREATE TABLE account_audit_log (
    audit_id INT AUTO_INCREMENT,
    account_id INT NOT NULL,
    operation_type VARCHAR(30),
    delta_amount DECIMAL(15,2),
    old_balance DECIMAL(15,2),
    new_balance DECIMAL(15,2),
    changed_by_app_user VARCHAR(100),
    changed_at DATETIME DEFAULT CURRENT_TIMESTAMP,
    PRIMARY KEY (audit_id),
    FOREIGN KEY (account_id) REFERENCES account(account_id)
);

CREATE TABLE loan (
    loan_id INT AUTO_INCREMENT,
    borrower_id INT NOT NULL,
    officer_id INT,
    mongodb_credit_profile_id VARCHAR(24),
    principal_amount DECIMAL(15,2) NOT NULL,
    annual_interest_rate DECIMAL(5,2) NOT NULL,
    term_months INT NOT NULL,
    status VARCHAR(30) NOT NULL,
    approval_date DATE,
    officer_comments VARCHAR(255),
    override_flag BOOLEAN DEFAULT FALSE,
    approved_at DATETIME,
    created_at DATETIME DEFAULT CURRENT_TIMESTAMP,
    updated_at DATETIME DEFAULT CURRENT_TIMESTAMP
        ON UPDATE CURRENT_TIMESTAMP,
    PRIMARY KEY (loan_id),
    FOREIGN KEY (borrower_id) REFERENCES users(user_id),
    FOREIGN KEY (officer_id) REFERENCES users(user_id),
    CHECK (principal_amount > 0),
    CHECK (term_months > 0)
);

CREATE TABLE collateral (
    asset_id INT AUTO_INCREMENT,
    loan_id INT NOT NULL,
    type VARCHAR(50) NOT NULL,
    estimated_value DECIMAL(15,2),
    valuation_date DATE,
    PRIMARY KEY (asset_id),
    FOREIGN KEY (loan_id) REFERENCES loan(loan_id)
);

CREATE TABLE loan_repayment (
    repayment_id INT AUTO_INCREMENT,
    loan_id INT NOT NULL,
```

```
    installment_number INT NOT NULL,
    principal_due DECIMAL(15,2) NOT NULL,
    interest_due DECIMAL(15,2) NOT NULL,
    total_due DECIMAL(15,2) NOT NULL,
    amount_paid DECIMAL(15,2) DEFAULT 0.00,
    due_date DATE NOT NULL,
    paid_at DATETIME,
    is_settled BOOLEAN DEFAULT FALSE,
    PRIMARY KEY (repayment_id),
    FOREIGN KEY (loan_id) REFERENCES loan(loan_id),
    UNIQUE (loan_id, installment_number)
);
```

6. Correct Order of Execution

Parent tables must exist before child tables that reference them. Use this order:

1. users
2. branch
3. account
4. beneficiary
5. security_log
6. bank_transaction
7. account_audit_log
8. loan
9. collateral
10. loan_repayment

7. MongoDB Credit Profile

CREDIT_PROFILE is shown separately as a MongoDB component in the ER diagram. It is intended for flexible credit-risk information such as borrower details, credit bureau payloads, financial statements, risk score, model version and underwriting status. The relational LOAN table keeps mongodb_credit_profile_id so a loan can be linked with its MongoDB credit profile.

For the relational-schema lab, focus on the SQL tables above. CREDIT_PROFILE is the NoSQL part of the hybrid project.

8. Viva Quick Revision

Question	Easy answer
Why is user_id inside ACCOUNT?	USER-Holds-ACCOUNT is 1:N, so the PK of USER goes to the N-side ACCOUNT as an FK.
Why are there two account foreign keys in TRANSACTION?	ACCOUNT participates twice: source of transaction and destination of transaction.
Why is total_outstanding_debt not stored?	It is a derived attribute and can be calculated from loan data.

Why are first_name and last_name stored separately?	name is a composite attribute in the ER diagram.
Why are borrower_id and officer_id both foreign keys to USER?	The same USER entity plays two roles: borrower and loan officer.
Why do Holds, Borrows and Maintains not get separate tables?	They are simple 1:N relationships and are represented directly using foreign keys.